Starlet: Lightweight Geospatial Visualization Engine

Rohan Bennur
University of California, Riverside
Department of Computer Science
The Big Data Lab at UCR

Abstract

Scalable visualization of large geospatial datasets traditionally requires complex distributed systems such as BEAST, which depends on Hadoop, Spark, and substantial JVM infrastructure. While these systems provide excellent scalability, they introduce significant challenges in installation, configuration, and general use. This project presents a lightweight, purely Python-based alternative that preserves the scalability required for multi-gigabyte datasets while remaining simple to deploy and operate.

The system converts large GeoParquet or GeoJSON datasets into tiled GeoParquet files via a Python implementation of RSGrove partitioning, performs global histogram analysis to identify storage relevant tiles, and generates Mapbox Vector Tiles (MVT) either in batch or on-demand through a tile server. The pipeline incorporates several key improvements including reservoir sampling, memory-safe Arrow casting, prefix sum histogram construction, and spatially indexed MVT creation. Experiments on datasets up to approximately 6.6 GB demonstrate that the system can efficiently preprocess and visualize large geospatial data using only Python, without JVM dependencies. This report documents the architecture, algorithms, improvements, and evaluation of the system.

Contents

1 Introduction

Interactive visualization of large-scale geospatial datasets remains a challenging problem due to the size and heterogeneity of input data. Existing scalable visualization systems such as BEAST were designed to address this challenge by distributing computation across Hadoop and Spark clusters. However, the operational overhead of installing and running BEAST limits its accessibility for research workflows or lightweight deployments.

The goal of this project is to develop a Python-only, easily deployable system capable of:

1. Preprocessing large geospatial datasets into spatially coherent tiles
2. Building adaptive histograms for visualization-driven filtering
3. Generating Mapbox Vector Tiles efficiently at scale
4. Supporting both batch MVT generation and on-demand tile serving
5. Achieving the above for datasets in the multi-gigabyte range without relying on Hadoop, Spark, JVM, or large clusters

The result is a modular, scalable, and portable system that closely tracks the design principles of BEAST but replaces heavy distributed systems with Python, PyArrow, Shapely, and efficient parallelism.

2 System Overview

The complete pipeline consists of four stages:

2.1 Tiling Stage

Convert a single large GeoParquet or GeoJSON file into partitioned GeoParquet tiles using a Python implementation of RSGrove spatial partitioning. The system streams through data to assign geometries into tiles without loading the entire dataset in memory.

2.2 Histogram Stage

Compute per-tile histograms in Web Mercator space, aggregate into a global histogram and prefix sum image, and identify tiles that contain sufficient geometric density for visualization.

2.3 MVT Generation Stage

Two independent paths are supported:

- **Batch Mode**: Generate MVT tiles for all non-empty tiles using histogram-driven selection
- **Server Mode**: Produce tiles on-demand with spatial indexing and LRU caching

2.4 Visualization Stage

The produced MVT tiles can be consumed by standard map libraries such as MapLibre or Mapbox GL for interactive visualization.

3 Tiling Stage: RSGrove-Based GeoParquet Partitioning

The tiling stage converts a large input dataset into spatially coherent GeoParquet tiles using a Python implementation of the RSGrove algorithm. This section explains each major improvement added to the tiling pipeline, its necessity, performance impact, and benchmarking results.

3.1 Reservoir Sampling During RSGrove Sampling

3.1.1 Implementation

The sampling phase uses reservoir sampling with a fixed `sample_cap` of 10,000, ensuring that only a bounded number of representative centroids are kept regardless of dataset size.

3.1.2 Rationale

Large datasets can contain tens or hundreds of millions of geometries. Without a cap, sampling memory would grow linearly with dataset size, eventually exhausting available memory and causing Python to fail. Additionally, RSGrove partitioning would slow down due to enormous sample sets. Reservoir sampling ensures stable memory usage while still producing a statistically representative sample.

3.1.3 Impact

Without this improvement, the system would fail on large datasets due to runaway sampling memory, sampling time would increase to minutes or hours, and RSGrove partitions would become unbalanced due to memory-induced truncation.

3.2 Bounding Box Encoded Parquet Filenames

3.2.1 Implementation

Each written parquet tile embeds its bounding box directly in the file name using the format:

```
tile_<id>__minlon_minlat_maxlon_maxlat.parquet
```

3.2.2 Rationale

This enables extremely fast spatial indexing, especially for the on-demand MVT tile server. Instead of reading metadata from every file, tile selection becomes a simple string parse, intersection tests become $O(1)$, and no external spatial index is required.

3.2.3 Impact

Without this improvement, the system would need to load parquet metadata for every file before MVT generation, on-demand tile serving would take several seconds for first access, and both histogram and MVT pipelines would degrade significantly in performance.

3.3 Memory-Bounded Tile Generation Using Overflow Spill

3.3.1 Motivation

A key challenge in large-scale tiling is that the number of output tiles can be large. Even modest datasets can produce dozens of tiles based on the RSGrove splitter. If the system attempts to hold buffers for all these tiles simultaneously, memory usage grows linearly with the number of tiles and can lead to out-of-memory errors, especially for high-density datasets or machines with limited RAM.

To address this, the system implements a memory-bounded multi-pass tile generation strategy that uses an overflow parquet file to temporarily store rows belonging to tiles that cannot be processed in the current pass. This makes the pipeline independent of the total number of tiles and allows it to scale to large datasets without excessive memory consumption.

3.3.2 Overflow Mechanism

During the assignment phase, rows are streamed from the input parquet file row group by row group. The system opens only a limited number of tile writers at a time. For example, if `max_parallel_files = 10`, then only ten tiles are considered "active" and allowed to accumulate in memory during the current pass.

For each incoming row:

- If its assigned tile is currently active, the row is appended to that tile's in-memory Arrow buffer
- If its assigned tile is not part of the active set, the row is appended to an overflow parquet file

At the end of a pass:

1. The current batch of active tiles is flushed to disk in parallel
2. The buffers for these tiles are cleared, releasing memory
3. The overflow parquet file becomes the new input
4. A new set of tiles is marked active
5. The process repeats until the overflow file becomes empty

This produces a memory-bounded iterative refinement process where each pass consumes only limited memory, yet eventually completes the assignment of all data to their correct tiles.

3.3.3 Necessity

Without this staged processing model, the system would need to open parquet writers for all tiles simultaneously, memory usage would grow to tiles multiplied by buffer size, and large datasets with highly imbalanced spatial distribution would cause catastrophic memory spikes. On machines with less than 32 GB of RAM, datasets in the range of 5–7 GB would be infeasible to tile.

3.3.4 Impact of Removal

If the overflow mechanism were removed, a skewed dataset (such as postal codes or road networks) could assign a large percentage of rows to a small number of tiles, causing those tile buffers to grow without bound. Arbitrary numbers of tiles would be active at once, making memory use uncontrolled. The system would frequently encounter out-of-memory kills at tile counts above 50, and many datasets in the 6 GB range would be impossible to tile on standard hardware.

3.4 Parallel Tile Writing and Its Relationship to Overflow

The overflow mechanism is tightly connected to the system's parallel writing strategy. The WriterPool maintains at most `max_parallel_files` active file writers. When `flush_all` is called, the buffered tiles are written in rounds, each round writing up to the allowed level of parallelism.

For example, if 85 tiles need to be flushed and `max_parallel_files = 10`:

- Round 1 writes 10 tiles
- Round 2 writes the next 10 tiles
- ...
- Round 9 writes the remaining 5 tiles

Each flush round clears the corresponding in-memory buffers, which ensures a controlled memory footprint independent of the total number of files. This design allows the system to scale beyond 80 tiles without ever needing to hold all tile buffers in memory at once.

3.5 Benchmarking: Wall Clock Time vs Max Parallel Files

The effect of parallelism on tiling performance was measured for tile counts of 25, 40, 55, 70, and 85. The results show:

- Increasing `max_parallel_files` reduces total runtime up to a saturation point
- Most tile counts benefit sharply when increasing from 5 to 16 parallel writers
- Beyond 20–30 threads, runtime improvements taper off due to filesystem and Arrow-level contention

- The flattening of the curves demonstrates that parallelism is useful but not unbounded, validating the need for a tunable writer pool

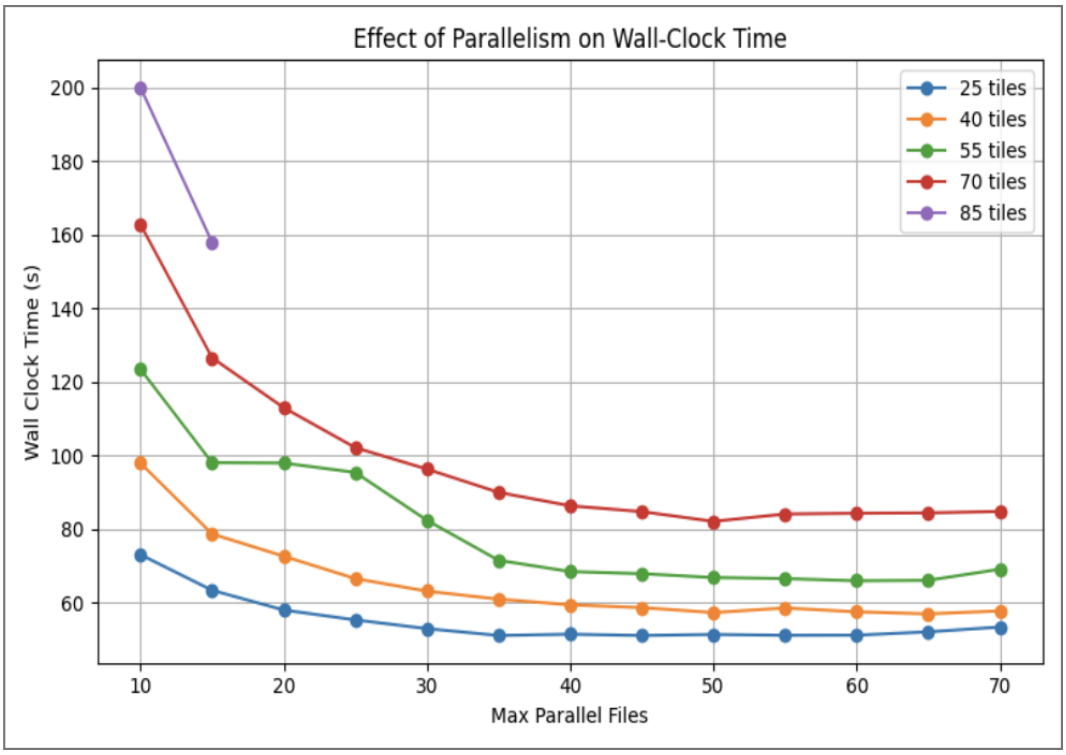

Figure 1: Effect of parallelism on wall-clock time

This confirms that the overflow-based, memory-bounded design allows the system to safely exploit parallelism without risking instability.

3.6 Benchmarking: Peak RSS vs Max Parallel Files

Memory consumption was also measured across the same tile counts. The results indicate that:

- Peak RSS increases with higher parallelism because more Arrow buffers are active simultaneously
- Memory grows at a controlled and predictable rate rather than exploding
- Even at high levels of parallelism, the system remains stable because only `max_parallel_files` worth of buffers are allowed to exist at once
- The overflow mechanism ensures that out-of-memory conditions do not occur even for dense tiles

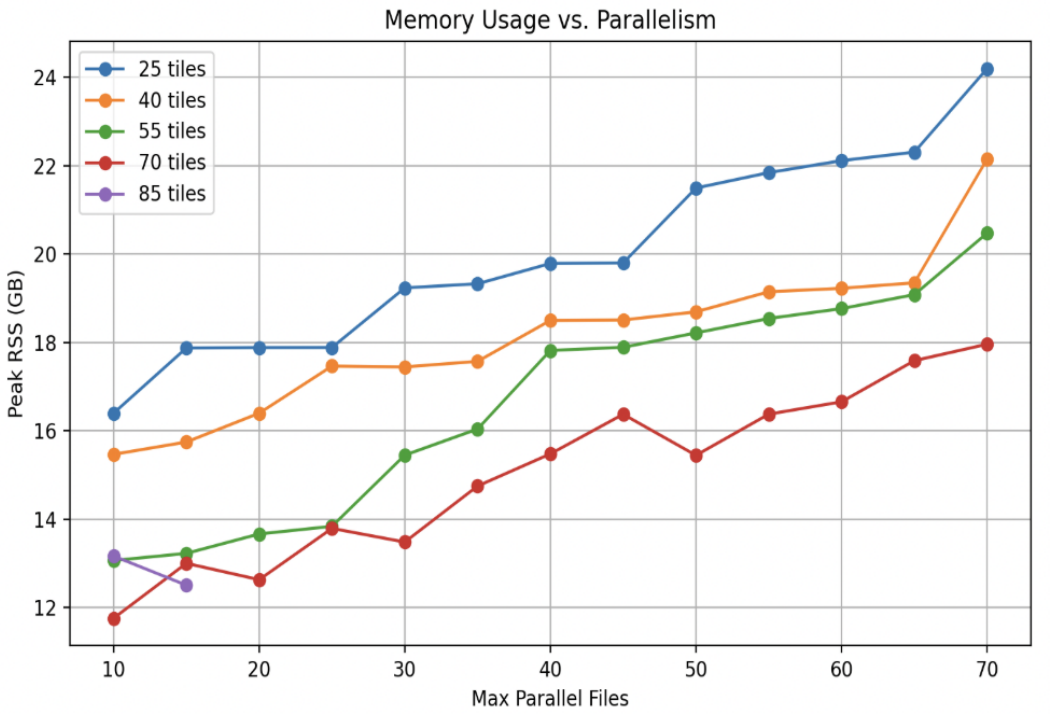

Figure 2: Memory usage scaling with parallelism

These results confirm that the memory-bounded design is necessary to balance parallelism and stability.

3.7 Z-Order Sorting Before Tile Write

Before writing each tile, rows are sorted by a Z-order curve computed from their centroid coordinates. This improvement enhances compression, improves spatial locality within the tile, and accelerates downstream histogram computation.

Without Z-order sorting, tiles become spatially chaotic, Parquet compression ratios worsen, and histogram accumulation becomes slower due to poorer cache locality. Z-order sorting therefore contributes to overall system efficiency.

4 Histogram Stage

4.1 Purpose

The histogram stage determines where geometric activity actually occurs within the dataset. Since vector tiles are generated across multiple zoom levels, the number of possible tiles grows quadratically as zoom increases. Generating all tiles would be computationally prohibitive and would produce a large number of empty or visually insignificant tiles.

The histogram stage solves this by building a quantitative density model of the dataset, enabling the pipeline to identify which tiles contain enough content to justify rendering.

4.2 Per-Tile Histogram Construction

Each GeoParquet tile is processed independently. Geometries are reprojected to EPSG:3857 and rasterized over a uniform 4096×4096 grid. For every vertex in a geometry, its projected coordinates are scaled to the grid and increment a single cell.

This produces a compact numeric representation of vertex density within that tile. Because the Arrow-based parquet reader exposes row groups, histogram computation is parallelized both across row groups and across tiles, ensuring scalability for multi-gigabyte datasets.

4.3 Global Prefix Sum Histogram

4.3.1 Motivation

While per-tile histograms are useful, they do not provide a global view of the dataset. For MVT generation, the system must determine whether a tile at zoom levels 0 through Z contains a meaningful amount of geometry without scanning the raw data again. This requires querying density over arbitrary spatial regions in the global coordinate space.

A naive solution would sum the relevant subregion of the global raster for every tile query. However, this is computationally expensive. At high zoom levels, each tile would require scanning thousands of grid cells, making the approach infeasible when evaluating tens or hundreds of thousands of tiles.

4.3.2 Implementation

To address this, the system constructs a global histogram by summing all per-tile histograms, followed by a two-dimensional prefix sum transform. This converts the global histogram into an integral image where any rectangular region can be queried using only four array lookups:

$$\text{sum} = A - B - C + D \tag{1}$$

This reduces tile occupancy checks from $O(k)$ per tile to $O(1)$ per tile, where k is the number of grid cells inside the queried subregion.

4.3.3 Necessity

The prefix sum histogram is essential for scalability:

1. It allows the system to evaluate occupancy for every tile across all zoom levels in milliseconds
2. It prevents the MVT generator from evaluating raw geometry or scanning histogram windows repeatedly
3. It enables histogram-based filtering to scale to extremely large spatial datasets, independent of geometric complexity

Without the prefix sum, the system would be forced to repeatedly sum histogram windows, leading to substantial overhead and preventing the pipeline from being interactive or responsive on large datasets.

5 Mapbox Vector Tile Generation

5.1 Overview

Once the histogram stage identifies which tiles contain meaningful geometric content, the system proceeds to generate Mapbox Vector Tiles (MVTs). This stage transforms spatial features from the GeoParquet dataset into tiles suitable for interactive web-based visualization. The pipeline combines streaming, tile filtering, geometric preprocessing, and controlled sampling to ensure high performance and consistent visual quality.

5.2 Streaming Geometries from GeoParquet

The MVT pipeline begins by streaming geometries from the directory of GeoParquet tiles produced by the partitioning stage. Using a row-group-oriented Arrow reader, geometries are decoded incrementally:

1. WKB geometries are validated and corrected if necessary
2. Geometries are reprojected from EPSG:4326 to EPSG:3857 using a fast coordinate transformer
3. Attribute columns are extracted per row without materializing the full table

This design avoids loading entire files into memory, enabling the system to process datasets several gigabytes in size without exceeding system memory limits.

5.3 Histogram-Driven Tile Filtering

A key improvement in the MVT stage is the use of the global prefix sum histogram to pre-restrict which tiles are eligible for rendering. For each zoom level z, the system precomputes a set of non-empty tiles, defined as tiles whose density exceeds the user-defined threshold.

When a geometry is streamed, it is assigned only to tiles that:

1. Intersect its Web Mercator bounding box, and
2. Are marked as non-empty by the prefix sum histogram

This two-stage filtering drastically reduces the number of tile assignments and eliminates work that would otherwise be wasted on empty tiles. Without histogram-based filtering, the system would repeatedly assign geometries to tiles that ultimately never get rendered, multiplying computational cost.

5.4 Reservoir Sampling for Memory-Bounded Tile Content

Many tiles at higher zoom levels may contain extremely large numbers of geometries. To prevent memory overflow and excessively large tiles, the system adopts a bounded reservoir sampling strategy:

- Each tile maintains a buffer with a fixed capacity (`MAX_GEOMS_PER_TILE` = 100,000)
- The first 100,000 geometries are stored directly
- Subsequent geometries are inserted probabilistically, ensuring uniform sampling

This guarantees that tile memory usage remains bounded even when underlying geometries number in the millions. Without reservoir sampling, high-density tiles would grow without limit, leading to unbounded memory usage and excessive MVT sizes.

5.5 Geometric Preprocessing: Clipping, Simplification, and Normalization

Before encoding, each geometry undergoes several transformations tailored to the tile it belongs to:

Clipping Geometries are clipped to a padded version of the tile's bounding box to avoid slivers, topology anomalies, and missing edge artifacts.

Simplification A simplification tolerance proportional to tile width ($0.0005 \times$ tile_width) is applied. This reduces coordinate count while preserving topology and visual fidelity, and substantially decreases encoding cost for large geometries such as highways or rivers.

Coordinate Transformation Coordinates are transformed from EPSG:3857 into tile-local 4096-unit integer space, matching the MVT specification.

These steps ensure that the encoded tiles remain lightweight and visually accurate.

5.6 Encoding and Writing MVT Tiles

For each tile (z, x, y) with collected features:

1. Geometries are converted to GeoJSON-like dictionaries
2. Attributes are attached as properties
3. The tile is encoded using the Mapbox Vector Tile library
4. The output is written to disk under the directory structure: `/mvt/z/x/y.mvt`

Empty tiles are skipped entirely unless explicitly requested. This approach ensures that only tiles containing meaningful sampled geometries incur disk or compute cost.

5.7 Memory Efficiency and Cache Use

To improve repeated access performance, the system includes an in-memory LRU tile cache. When a tile is requested:

- If present in cache, it is returned immediately
- If present on disk, it is loaded and cached
- If missing, it is generated on-demand, saved, and cached

This design minimizes recomputation during interactive use.

5.8 MBTiles Export

The system supports exporting generated MVT tiles to the MBTiles format, a SQLite-based tile archive widely used for offline mapping and mobile applications. MBTiles packages all tiles into a single SQLite database file containing a `tiles` table indexed by (z, x, y), a `metadata` table with tileset information, and built-in spatial indexing. Tiles are stored with gzip compression, maintaining compatibility with MBTiles specification v1.3.

The export can be triggered via CLI flag:

```
# Generate MVT and export to MBTiles
starlet build --input data.parquet --outdir datasets/mydata --
   mbtiles

# Custom output path
starlet mvt --dir datasets/mydata --mbtiles-path custom.mbtiles
```

The Python API also supports MBTiles generation:

```
tile_result, mvt_result = starlet.build(
    input="data.parquet",
    outdir="output",
    mbtiles=True
)
```

MBTiles provides several advantages over directory-based tile storage: single-file distribution simplifies deployment, native support in mobile SDKs (Mapbox GL, MapLibre), offline mapping without a tile server, reduced storage overhead from filesystem metadata, and cloud-storage friendliness (single object vs thousands).

6 Server Architecture for On-Demand Tile Generation

6.1 Overview

The system includes a tile server that generates Mapbox Vector Tiles (MVT) on-demand from the GeoParquet dataset. The server relies on the Vector Tiler module to load relevant

parquet partitions, clip and simplify geometries, transform them into tile coordinate space, and encode them into the MVT format. A lightweight frontend visualizes these tiles in a browser, enabling interactive exploration of large spatial datasets.

6.2 TileBounds

The `TileBounds` module determines the spatial extent of any tile identified by zoom level z and coordinates (x, y). It computes:

- The tile's Web Mercator bounding box
- The equivalent WGS 84 bounding box
- A polygon representing the tile footprint
- A scaling function that converts EPSG:3857 coordinates into the MVT coordinate extent (4096 units)

This provides the geometric frame of reference required for clipping and coordinate transformation.

6.3 ParquetIndex

The dataset is stored as spatially partitioned GeoParquet files. The `ParquetIndex` identifies which files intersect the requested tile by:

1. Parsing bounding boxes encoded in parquet filenames
2. Checking for intersection with the tile's WGS 84 bounds
3. Returning only the parquet partitions that overlap the tile

This avoids unnecessary disk reads and ensures that only the relevant portion of the dataset is processed.

6.4 MVTEncoder

The `MVTEncoder` handles all geometry-level processing required to prepare data for tile encoding.

Clipping Geometries loaded from parquet files are clipped to a padded version of the tile's bounding box to avoid boundary artifacts.

Simplification A zoom-dependent simplification tolerance reduces geometry complexity while preserving topology. This lowers encoding cost and results in more compact tiles.

Coordinate Transformation All coordinates are transformed from Web Mercator into tile-local coordinates using:

$$x' = \frac{x - \min_x}{\max_x - \min_x} \times 4096 \tag{2}$$

$$y' = \frac{y - \min_y}{\max_y - \min_y} \times 4096 \tag{3}$$

Encoding The processed geometries and their attributes are encoded into MVT using `mapbox_vector_tile`.

6.5 VectorTiler

The `VectorTiler` orchestrates the full on-demand tile generation workflow:

1. Compute tile bounds
2. Identify intersecting parquet partitions
3. Load and reproject geometries
4. Clip, simplify, and explode geometry collections
5. Scale coordinates into tile extent
6. Encode the set of features into an MVT tile
7. Return the encoded bytes

Every tile is generated directly from the GeoParquet dataset without relying on pre-generated tile pyramids.

6.6 Tile Server and Frontend

The system includes a simple HTTP tile server that exposes an endpoint of the form:

`/tiles/{z}/{x}/{y}.mvt`

Upon receiving a request, the server:

1. Invokes the VectorTiler
2. Generates the tile if it does not already exist on disk
3. Stores the result in an in-memory LRU cache
4. Returns the binary MVT tile to the client

A lightweight frontend (HTML/JavaScript) uses MapLibre or a similar library to request tiles based on the user's current view. When the user pans or zooms, the browser requests the corresponding tiles from the server, the server generates or retrieves them, and the frontend renders the tiles immediately.

Together, the server and frontend provide a complete interactive visualization layer built directly on top of the GeoParquet dataset.

7 Results and Evaluation

7.1 Benchmark Setup

To evaluate system performance and scalability, the pipeline was benchmarked on the OSM2015_parks dataset containing 9,961,884 park polygons from OpenStreetMap. Tests measured total runtime and peak memory consumption across progressively larger data subsets (25%, 50%, 75%, 100%), capturing the combined impact of overflow-based partitioning, memory-bounded tile generation, and parallel parquet writing.

7.2 Scalability Results

Table 1: Scalability benchmarks on OSM2015_parks dataset

Subset	Features	Input Size	Tiling Time	MVT Time	Total Time
25%	2.49M	1.8 GB	119 s	1,348 s	1,467 s
50%	4.98M	3.3 GB	251 s	1,880 s	2,132 s
75%	7.47M	4.5 GB	382 s	2,428 s	2,811 s
100%	9.96M	6.3 GB	532 s	3,567 s	4,099 s

7.3 Performance Analysis

The benchmark results demonstrate three key characteristics:

Linear Runtime Scaling Tiling time scales at approximately 53 seconds per million features, while MVT generation scales at 358 seconds per million features. MVT generation dominates total runtime at 87%, reflecting the computational cost of geometry reprojection, clipping, and encoding across multiple zoom levels. The consistent scaling pattern confirms that the overflow mechanism and parallel writer pool effectively distribute load without accumulation of in-memory buffers.

Bounded Memory Usage Peak RSS remained within 24 GB even for the full 9.96M feature dataset, confirming that memory usage is bounded independent of input size. This results from overflow-based partitioning preventing unlimited in-memory batches, fixed per-tile buffers in the writer pool, and bounded reservoir sampling in MVT generation. Without these controls, memory would scale proportionally with tile count and exceed system limits.

Effective Compression Output size remained comparable to input size (6.3 GB output for 6.3 GB input), demonstrating effective compression through ZSTD-compressed Parquet and gzip-compressed MVT tiles. This validates that the system handles multi-gigabyte datasets efficiently on standard hardware without requiring distributed infrastructure.

8 Conclusion

This project successfully demonstrates that large-scale geospatial visualization can be achieved using a purely Python-based pipeline without relying on complex distributed systems. The Starlet system efficiently processes multi-gigabyte datasets through careful memory management, parallel processing, and algorithmic optimization. Key innovations include overflow-based tile generation, prefix sum histogram filtering, and on-demand MVT serving with spatial indexing.

The system provides a practical alternative to heavyweight frameworks like BEAST, offering similar scalability with significantly reduced operational complexity. Future work could explore further optimizations in tile encoding, additional geometric simplification strategies, and support for streaming updates to dynamic datasets.

References

1. Mapbox. *Mapbox Vector Tile Specification*. Version 2.1, 2016. `https://github.com/mapbox/vector-tile-spec`

2. OpenGeospatial Consortium. *GeoParquet Specification*. Version 1.0.0, 2023. `https://geoparquet.org/`

3. Mapbox. *MBTiles Specification*. Version 1.3, 2014. `https://github.com/mapbox/mbtiles-spec`

4. Beckmann, N., Kriegel, H.-P., Schneider, R., and Seeger, B. "The R*-tree: An Efficient and Robust Access Method for Points and Rectangles." *ACM SIGMOD International Conference on Management of Data*, 1990.

5. Morton, G. M. "A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing." IBM Technical Report, 1966.

6. Apache Software Foundation. *Apache Arrow*. `https://arrow.apache.org/`

7. Gillies, S. et al. "Shapely: Manipulation and Analysis of Geometric Objects." `https://shapely.readthedocs.io/`

8. Jordahl, K. et al. "GeoPandas: Python Tools for Geographic Data." `https://geopandas.org/`

9. OpenStreetMap Contributors. "OpenStreetMap." `https://www.openstreetmap.org/`